

✓ Khyati Sharma, Jigyasa Soni & Vishaka Sharma

Aqua-Net: A Deep Dive into Aquatic Life

✓ Importing Required Modules

```
import pandas as pd
import numpy as np
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
import time
import matplotlib.pyplot as plt
import cv2
import seaborn as sns
sns.set_style('darkgrid')
import shutil
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Dense, Activation, Dropout, Conv2D, MaxPooling2D, BatchNormalization
from tensorflow.keras.optimizers import Adam, Adamax
from tensorflow.keras.metrics import categorical_crossentropy
from tensorflow.keras import regularizers
from tensorflow.keras.models import Model
from tensorflow.keras import backend as K
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping, ModelCheckpoint
import time
import numpy as np
from tqdm import tqdm
from sklearn.metrics import f1_score
import sys
if not sys.warnoptions:
    import warnings
    warnings.simplefilter("ignore")

import os
os.chdir("/content/drive/MyDrive/Final_Project_Deep_Learning")

# from google.colab import drive
# drive.mount('/content/drive')

Mounted at /content/drive

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

# !unzip "/content/drive/MyDrive/Colab Notebooks/Final_Project_Deep_Learning/three_class_data.zip"

import os

def count_images_in_classes(data_dir):
    """
    Count the number of images in each class for test, train, and val folders.

    :param data_dir: Path to the main dataset directory containing 'train', 'test', 'val' folders.
    :return: Dictionary with counts for each folder and class.
    """
    results = {}
```

```

# Iterate through 'train', 'test', 'val' directories
for folder in ['train', 'test', 'val']:
    folder_path = os.path.join(data_dir, folder)

    if not os.path.exists(folder_path):
        print(f"Folder '{folder}' does not exist in '{data_dir}'. Skipping...")
        continue

    class_counts = {}
    for class_name in os.listdir(folder_path):
        class_path = os.path.join(folder_path, class_name)

        if os.path.isdir(class_path):
            # Count image files in the class folder
            num_images = len([f for f in os.listdir(class_path) if os.path.isfile(os.path.join(class_path, f))])
            class_counts[class_name] = num_images

    results[folder] = class_counts

return results

def print_results(results):
    """
    Print the results in a readable format.

    :param results: Dictionary with counts for each folder and class.
    """
    for folder, class_counts in results.items():
        print(f"\n{folder.upper()} SET:")
        for class_name, count in class_counts.items():
            print(f"  {class_name}: {count} images")

# Main directory containing 'train', 'test', 'val'
data_directory = "/content/drive/MyDrive/Final_Project_Deep_Learning/split_data"

# Count images in each class
results = count_images_in_classes(data_directory)

# Print the results
print_results(results)

```



```

TRAIN SET:
Turtle_Tortoise: 1332 images
Dolphin: 547 images
Jelly Fish: 591 images

TEST SET:
Turtle_Tortoise: 286 images
Dolphin: 118 images
Jelly Fish: 127 images

VAL SET:
Turtle_Tortoise: 285 images
Dolphin: 117 images
Jelly Fish: 127 images

```

```

# plot training data distribution
import os
import matplotlib.pyplot as plt

def count_images_per_class(dataset_path):
    class_image_counts = {}
    for class_name in os.listdir(dataset_path):
        class_path = os.path.join(dataset_path, class_name)
        if os.path.isdir(class_path):
            image_files = [f for f in os.listdir(class_path) if os.path.isfile(os.path.join(class_path, f))]
            class_image_counts[class_name] = len(image_files)

    return class_image_counts

def plot_image_distribution(image_counts):
    classes = list(image_counts.keys())
    counts = list(image_counts.values())

    plt.figure(figsize=(12, 6))
    plt.bar(classes, counts, color='skyblue')

```

```
plt.xlabel('Classes')
plt.ylabel('Number of Images')
plt.title('Image Distribution Across Classes')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
dataset_path = "/content/drive/MyDrive/Final_Project_Deep_Learning/split_data/train"
image_counts = count_images_per_class(dataset_path)
for class_name, count in image_counts.items():
    print(f'Class '{class_name}': {count} images")
plot_image_distribution(image_counts)
```

```
→ Class 'Turtle_Tortoise': 1332 images
Class 'Dolphin': 547 images
Class 'Jelly Fish': 591 images
```



```
import os
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import load_img

# Define the path to the train folder
train_folder = '/content/drive/MyDrive/Final_Project_Deep_Learning/split_data/train'

# Get the class names (subfolder names)
class_names = sorted(os.listdir(train_folder))

# Initialize a figure to display images
plt.figure(figsize=(15, 10))

# Loop through each class and display one image
for i, class_name in enumerate(class_names):
    # Get the path to the class folder
    class_folder = os.path.join(train_folder, class_name)

    # Get a list of all image files in the class folder
    image_files = [f for f in os.listdir(class_folder) if os.path.isfile(os.path.join(class_folder, f))]

    if image_files: # Check if there are images in the folder
        # Get the path to the first image
        image_path = os.path.join(class_folder, image_files[0])

        # Load the image
        img = load_img(image_path)
```

```
# Add the image to the plot
plt.subplot(len(class_names) // 5 + 1, 5, i + 1)
plt.imshow(img)
plt.axis('off')
plt.title(class_name)

# Adjust the layout and show the plot
plt.tight_layout()
plt.show()
```



Dolphin



Jelly Fish



Turtle_Tortoise



```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os

def create_generators(main_dir, img_size, batch_size):

    # Paths to subdirectories
    train_dir = os.path.join(main_dir, 'train')
    val_dir = os.path.join(main_dir, 'val')
    test_dir = os.path.join(main_dir, 'test')

    # ImageDataGenerator for rescaling
    datagen = ImageDataGenerator(rescale=1.0 / 255)

    # Train generator
    print("Creating train generator...")
    train_gen = datagen.flow_from_directory(
        directory=train_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical', # Use 'binary' for binary classification
        shuffle=True,
        color_mode='rgb'
    )

    # Validation generator
    print("Creating validation generator...")
    valid_gen = datagen.flow_from_directory(
        directory=val_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical',
        shuffle=False,
        color_mode='rgb'
    )

    # Test generator
    print("Creating test generator...")
    test_gen = datagen.flow_from_directory(
        directory=test_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical',
        shuffle=False,
        color_mode='rgb'
    )

    # Retrieve class labels
    classes = list(train_gen.class_indices.keys())
    print(f"Classes found: {classes}")

    return train_gen, valid_gen, test_gen, classes
```

```
# Example usage
main_dir = "/content/drive/MyDrive/Final_Project_Deep_Learning/split_data" # Main folder containing 'train', 'val', 'test'
img_size = (224, 224) # Image size for resizing
batch_size = 32 # Batch size for generators
```

```
train_gen, valid_gen, test_gen, classes = create_generators(main_dir, img_size, batch_size)
```

```
# Example: Display generator details
print(f"Number of classes: {len(classes)}")
print(f"Class labels: {classes}")
```

```
→ Creating train generator...
Found 2470 images belonging to 3 classes.
Creating validation generator...
Found 529 images belonging to 3 classes.
Creating test generator...
Found 531 images belonging to 3 classes.
Classes found: ['Dolphin', 'Jelly Fish', 'Turtle_Tortoise']
Number of classes: 3
Class labels: ['Dolphin', 'Jelly Fish', 'Turtle_Tortoise']
```

```
from keras.layers import Dropout, Dense, Flatten, BatchNormalization, Conv2D, MaxPooling2D, GlobalAveragePooling2D
from tensorflow.keras.optimizers import SGD
from keras import layers
from keras.models import Sequential
from keras.callbacks import EarlyStopping
```

```
early_stop=EarlyStopping(monitor='val_loss',patience=3)
strategy = tf.distribute.MirroredStrategy()
```

```
with strategy.scope():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same', input_shape=(224, 224, 3)))
    model.add(BatchNormalization())
```

```
# model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))
#model.add(Dropout(0.2))
```

```
model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
model.add(BatchNormalization())
# model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
# model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))
# model.add(Dropout(0.3))
model.add(Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
model.add(BatchNormalization())
# model.add(Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
# model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))
# model.add(Dropout(0.4))
model.add(Flatten())
model.add(Dense(128, activation='relu', kernel_initializer='he_uniform'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(3, activation='softmax'))
```

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```
→ /usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape` to `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.callbacks import ModelCheckpoint
```

```
# Define callbacks
callbacks = [
    ModelCheckpoint(
```

```
        filepath='best_model.keras', # Save the model in .h5 format
        monitor='val_loss',
        save_best_only=True, # Save only the best model
        mode='min', # Minimizing the validation loss
        verbose=1 # Print updates during training
    ),
    EarlyStopping(
        monitor='val_loss', # Monitor validation loss
        patience=5, # Stop training after 5 epochs of no improvement
        verbose=1, # Print when stopping
        mode='min' # Minimizing the validation loss
    )
]
print(model.summary())

epochs = 40
history=model.fit(x=train_gen, epochs=epochs, verbose=1, validation_data=valid_gen,
                  validation_steps=None, shuffle=False, initial_epoch=0)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
batch_normalization_1 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_3 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten (Flatten)	(None, 100352)	0
dense (Dense)	(None, 128)	12,845,184
batch_normalization_4 (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 12,940,355 (49.36 MB)

Trainable params: 12,939,587 (49.36 MB)

Non-trainable params: 768 (3.00 KB)

None

```
/usr/local/lib/python3.10/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your `PyDataset` class
self._warn_if_super_not_called()
```

Epoch 1/40

78/78 ————— 28s 238ms/step - accuracy: 0.7723 - loss: 0.7952 - val_accuracy: 0.6352 - val_loss: 0.7377

Epoch 2/40

78/78 ————— 21s 265ms/step - accuracy: 0.8357 - loss: 0.4678 - val_accuracy: 0.7505 - val_loss: 0.6710

Epoch 3/40

78/78 ————— 17s 221ms/step - accuracy: 0.8760 - loss: 0.3699 - val_accuracy: 0.8242 - val_loss: 0.4695

Epoch 4/40

78/78 ————— 21s 263ms/step - accuracy: 0.8849 - loss: 0.3377 - val_accuracy: 0.8582 - val_loss: 0.3603

Epoch 5/40

78/78 ————— 19s 237ms/step - accuracy: 0.8999 - loss: 0.3002 - val_accuracy: 0.8374 - val_loss: 0.4734

Epoch 6/40

78/78 ————— 18s 221ms/step - accuracy: 0.9056 - loss: 0.2606 - val_accuracy: 0.8771 - val_loss: 0.3408

Epoch 7/40

78/78 ————— 21s 236ms/step - accuracy: 0.9060 - loss: 0.2466 - val_accuracy: 0.8885 - val_loss: 0.2956

Epoch 8/40

78/78 ————— 18s 228ms/step - accuracy: 0.9411 - loss: 0.1855 - val_accuracy: 0.8922 - val_loss: 0.3102

Epoch 9/40

78/78 ————— 20s 225ms/step - accuracy: 0.9316 - loss: 0.1949 - val_accuracy: 0.8941 - val_loss: 0.2970

Epoch 10/40

78/78 ————— 18s 224ms/step - accuracy: 0.9455 - loss: 0.1681 - val_accuracy: 0.9055 - val_loss: 0.2833

Epoch 11/40

78/78 ————— 20s 257ms/step - accuracy: 0.9621 - loss: 0.1086 - val_accuracy: 0.8639 - val_loss: 0.4018

Epoch 12/40

78/78 ————— 20s 254ms/step - accuracy: 0.9562 - loss: 0.1217 - val_accuracy: 0.9017 - val_loss: 0.3498

Epoch 13/40

78/78 ————— 18s 226ms/step - accuracy: 0.9687 - loss: 0.0963 - val_accuracy: 0.9017 - val_loss: 0.3069

Epoch 14/40

78/78 ————— 18s 225ms/step - accuracy: 0.9535 - loss: 0.1292 - val_accuracy: 0.8847 - val_loss: 0.3669

Epoch 15/40

78/78 ————— 19s 238ms/step - accuracy: 0.9721 - loss: 0.0937 - val_accuracy: 0.9055 - val_loss: 0.2911

Epoch 16/40

78/78 ————— 19s 237ms/step - accuracy: 0.9813 - loss: 0.0594 - val_accuracy: 0.8941 - val_loss: 0.3468

Epoch 17/40

78/78 ————— 18s 225ms/step - accuracy: 0.9815 - loss: 0.0553 - val_accuracy: 0.8998 - val_loss: 0.3175

Epoch 18/40

78/78 ————— 18s 229ms/step - accuracy: 0.9748 - loss: 0.0731 - val_accuracy: 0.8960 - val_loss: 0.4130

Epoch 19/40

78/78 ————— 20s 257ms/step - accuracy: 0.9259 - loss: 0.2422 - val_accuracy: 0.8809 - val_loss: 0.3691

Epoch 20/40

78/78 ————— 17s 221ms/step - accuracy: 0.9541 - loss: 0.1347 - val_accuracy: 0.9055 - val_loss: 0.3523

```

Epoch 21/40
78/78 ————— 18s 226ms/step - accuracy: 0.9675 - loss: 0.0910 - val_accuracy: 0.9187 - val_loss: 0.3131
Epoch 22/40
78/78 ————— 18s 231ms/step - accuracy: 0.9703 - loss: 0.0762 - val_accuracy: 0.9244 - val_loss: 0.2845
Epoch 23/40
78/78 ————— 20s 254ms/step - accuracy: 0.9799 - loss: 0.0570 - val_accuracy: 0.9187 - val_loss: 0.3073
Epoch 24/40
78/78 ————— 18s 224ms/step - accuracy: 0.9945 - loss: 0.0325 - val_accuracy: 0.9206 - val_loss: 0.3072
Epoch 25/40
78/78 ————— 17s 221ms/step - accuracy: 0.9828 - loss: 0.0546 - val_accuracy: 0.9187 - val_loss: 0.3412
Epoch 26/40
78/78 ————— 18s 226ms/step - accuracy: 0.9910 - loss: 0.0329 - val_accuracy: 0.9130 - val_loss: 0.3418
Epoch 27/40
78/78 ————— 21s 264ms/step - accuracy: 0.9895 - loss: 0.0364 - val_accuracy: 0.9093 - val_loss: 0.3311
Epoch 28/40
78/78 ————— 20s 254ms/step - accuracy: 0.9913 - loss: 0.0312 - val_accuracy: 0.9130 - val_loss: 0.3367
Epoch 29/40
78/78 ————— 18s 228ms/step - accuracy: 0.9824 - loss: 0.0639 - val_accuracy: 0.8223 - val_loss: 0.7007
Epoch 30/40
78/78 ————— 20s 215ms/step - accuracy: 0.9324 - loss: 0.2000 - val_accuracy: 0.8280 - val_loss: 0.5831
Epoch 31/40
78/78 ————— 18s 224ms/step - accuracy: 0.9637 - loss: 0.0920 - val_accuracy: 0.8847 - val_loss: 0.3772
Epoch 32/40
78/78 ————— 19s 235ms/step - accuracy: 0.9835 - loss: 0.0515 - val_accuracy: 0.8998 - val_loss: 0.3491
Epoch 33/40
78/78 ————— 20s 233ms/step - accuracy: 0.9874 - loss: 0.0397 - val_accuracy: 0.9093 - val_loss: 0.3079
Epoch 34/40
78/78 ————— 20s 256ms/step - accuracy: 0.9908 - loss: 0.0350 - val_accuracy: 0.9149 - val_loss: 0.3214
Epoch 35/40
78/78 ————— 20s 261ms/step - accuracy: 0.9911 - loss: 0.0357 - val_accuracy: 0.9112 - val_loss: 0.3253
Epoch 36/40
78/78 ————— 20s 254ms/step - accuracy: 0.9924 - loss: 0.0245 - val_accuracy: 0.9036 - val_loss: 0.3334
Epoch 37/40
78/78 ————— 18s 227ms/step - accuracy: 0.9886 - loss: 0.0416 - val_accuracy: 0.9036 - val_loss: 0.3907
Epoch 38/40
78/78 ————— 18s 222ms/step - accuracy: 0.9895 - loss: 0.0414 - val_accuracy: 0.9112 - val_loss: 0.3725
Epoch 39/40
78/78 ————— 20s 259ms/step - accuracy: 0.9789 - loss: 0.0672 - val_accuracy: 0.9112 - val_loss: 0.3678
Epoch 40/40
78/78 ————— 18s 231ms/step - accuracy: 0.9936 - loss: 0.0274 - val_accuracy: 0.9055 - val_loss: 0.3858

```

```
import matplotlib.pyplot as plt
```

```
# Assuming `history` is the output from model.fit
```

```
def plot_training_history(history):
```

```
    # Get training and validation loss
```

```
    train_loss = history.history['loss']
```

```
    val_loss = history.history['val_loss']
```

```
    # Create a plot
```

```
    plt.figure(figsize=(8, 6))
```

```
    plt.plot(train_loss, label='Training Loss', color='blue', linestyle='-')
```

```
    plt.plot(val_loss, label='Validation Loss', color='orange', linestyle='--')
```

```
    plt.xlabel('Epochs')
```

```
    plt.ylabel('Loss')
```

```
    plt.title('Training and Validation Loss')
```

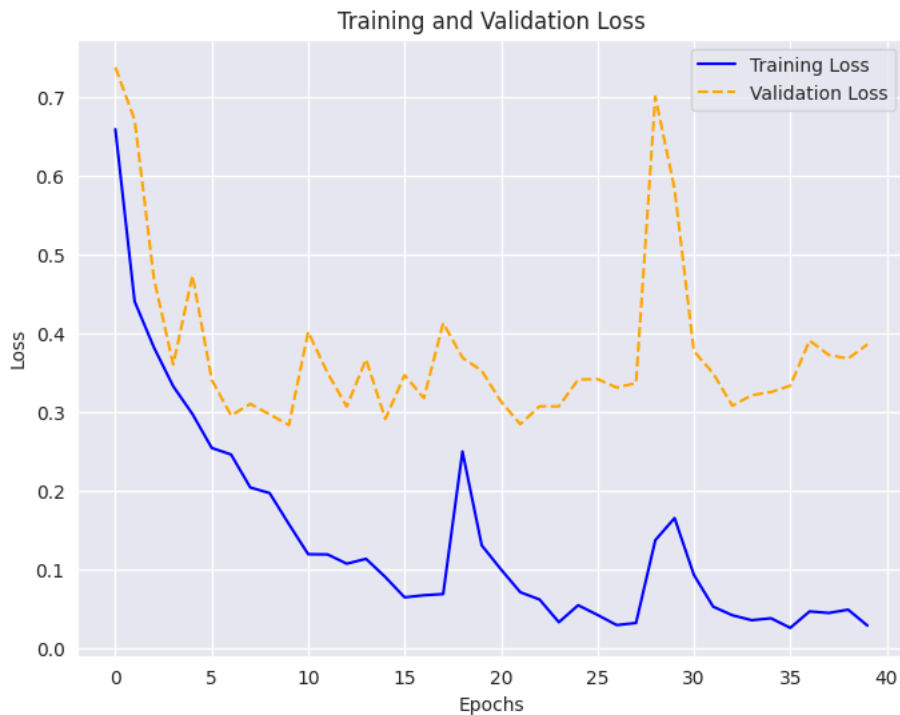
```
    plt.legend()
```

```
    plt.grid(True)
```

```
    plt.show()
```

```
# Call the function to plot
```

```
plot_training_history(history)
```



```
def evaluate(test_gen):
    y_pred= []
    error_list=[]
    error_pred_list = []
    y_true=test_gen.labels
    classes=list(test_gen.class_indices.keys())
    class_count=len(classes)
    errors=0
    preds=model.predict(test_gen, verbose=1)
    tests=len(preds)
    for i, p in enumerate(preds):
        file=test_gen filenames[i]
        pred_index=np.argmax(p)
        true_index=test_gen.labels[i] # labels are integer values
        if pred_index != true_index: # a misclassification has occurred
            errors=errors + 1
            file=test_gen.filenames[i]
            error_class=classes[pred_index]
            t=(file, error_class)
            error_list.append(t)
    y_pred.append(pred_index)

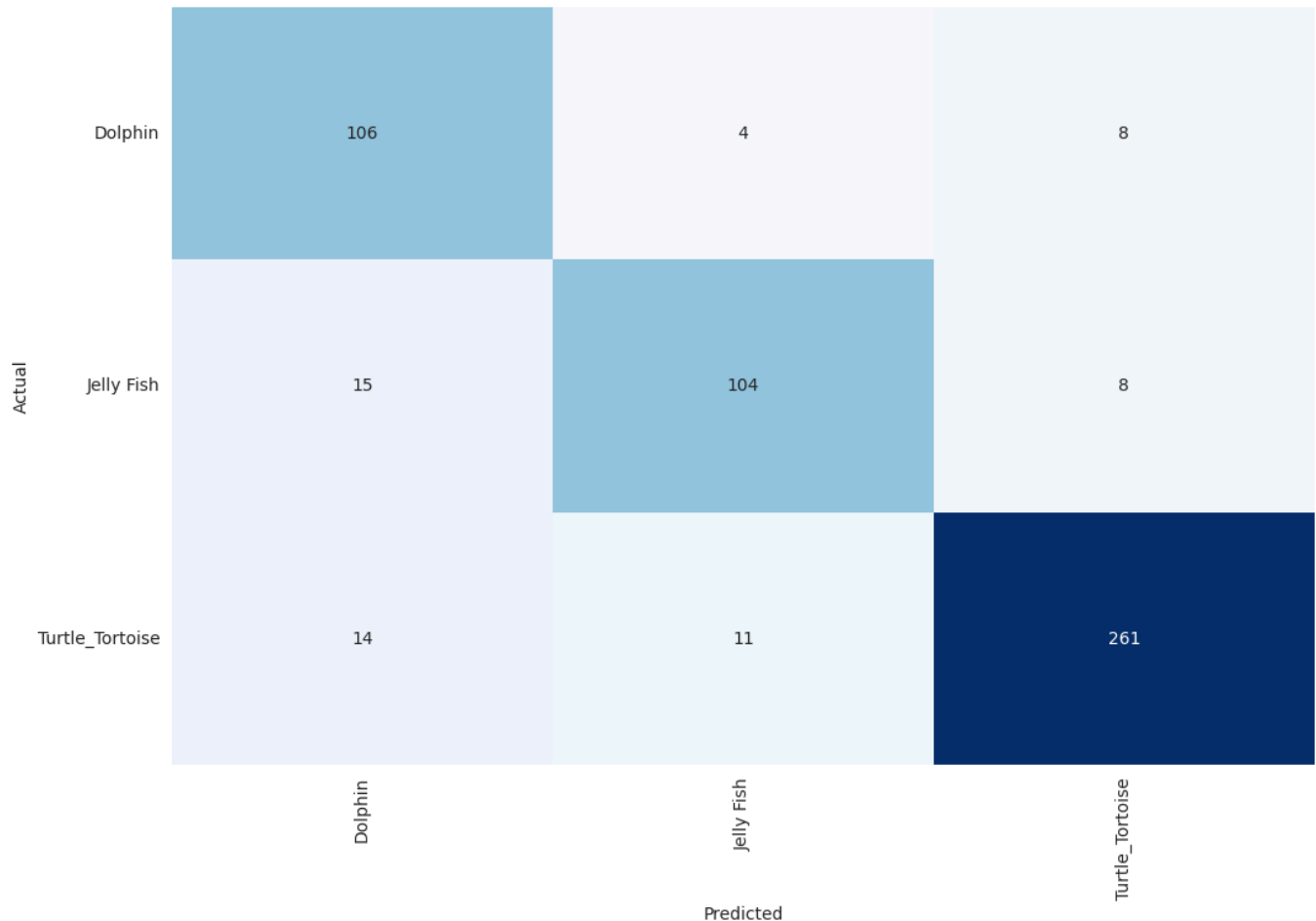
    acc=( 1-errors/tests) * 100
    msg=f'there were {errors} errors in {tests} tests for an accuracy of {acc:6.2f}'
    print(msg) # cyan foreground
    ypred=np.array(y_pred)
    ytrue=np.array(y_true)
    f1score=f1_score(ytrue, ypred, average='weighted')* 100
    if class_count <=30:
        cm = confusion_matrix(ytrue, ypred )
        # plot the confusion matrix
        plt.figure(figsize=(8, 6))
        sns.heatmap(cm, annot=True, vmin=0, fmt='g', cmap='Blues', cbar=False)
        plt.xticks(np.arange(class_count)+.5, classes, rotation=90)
        plt.yticks(np.arange(class_count)+.5, classes, rotation=0)
        plt.xlabel("Predicted")
        plt.ylabel("Actual")
        plt.title("Confusion Matrix")
        plt.show()
    clr = classification_report(y_true, y_pred, target_names=classes, digits= 4) # create classification report
    print("Classification Report:\n-----\n", clr)
    return errors, tests, error_list, f1score
```

✓ Evaluating The Model

```
errors, tests, error_list, f1score =evaluate(test_gen)
```

17/17 7s 386ms/step
there were 60 errors in 531 tests for an accuracy of 88.70

Confusion Matrix



Classification Report:

	precision	recall	f1-score	support
Dolphin	0.7852	0.8983	0.8379	118
Jelly Fish	0.8739	0.8189	0.8455	127
Turtle_Tortoise	0.9422	0.9126	0.9272	286
accuracy			0.8870	531
macro avg	0.8671	0.8766	0.8702	531
weighted avg	0.8910	0.8870	0.8878	531

```
#saving the trained model
working_dir=r'/content/drive/MyDrive/Colab Notebooks/Final_Project_Deep_Learning/'
subject='sea animals'
save_id=f'{subject}-{f1score:5.2f}.h5'
model_save_loc=os.path.join(working_dir, save_id)
model.save(model_save_loc)
msg= f'model was saved as {model_save_loc}'
print(msg)
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend you use the Keras 2.x `save\_model` function instead. Your model was saved as /content/drive/MyDrive/Colab Notebooks/Final_Project_Deep_Learning/sea animals-88.78.h5

Start coding or [generate](#) with AI.

✓ Generating Augmented Data and Training the Model Using Data Augmentation

Data augmentation so that each class contains equal number of images.

Technique use are :

- Horizontal flip = True
- rotation_range = 20
- width_shift_range =
- height_shift_range=0.2
- zoom_range = 0.2

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os

def create_generators(main_dir, img_size, batch_size):
    """
    Create ImageDataGenerator instances for train, validation, and test datasets with augmentations for the train data.

    Args:
        main_dir (str): Path to the main directory containing 'train', 'val', and 'test' subdirectories.
        img_size (tuple): Target size for resizing images, e.g., (224, 224).
        batch_size (int): Batch size for the generators.

    Returns:
        tuple: train_gen, valid_gen, test_gen, classes
    """
    # Paths to subdirectories
    train_dir = os.path.join(main_dir, 'train')
    val_dir = os.path.join(main_dir, 'val')
    test_dir = os.path.join(main_dir, 'test')

    # ImageDataGenerator for training with augmentations
    train_datagen = ImageDataGenerator(
        rescale=1.0 / 255, # Rescaling
        horizontal_flip=True, # Horizontal flip
        rotation_range=20, # Random rotation
        width_shift_range=0.2, # Random width shift
        height_shift_range=0.2, # Random height shift
        zoom_range=0.2 # Random zoom
    )

    # ImageDataGenerator for validation and test data (only rescaling)
    datagen = ImageDataGenerator(rescale=1.0 / 255)

    # Train generator with augmentations
    print("Creating train generator...")
    train_gen = train_datagen.flow_from_directory(
        directory=train_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical', # Use 'binary' for binary classification
        shuffle=True,
        color_mode='rgb'
    )

    # Validation generator (no augmentation, only rescaling)
    print("Creating validation generator...")
    valid_gen = datagen.flow_from_directory(
        directory=val_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical',
        shuffle=False, # No shuffling in validation data
        color_mode='rgb'
    )

    # Test generator (no augmentation, only rescaling)
    print("Creating test generator...")
    test_gen = datagen.flow_from_directory(
```

```

        directory=test_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical',
        shuffle=False, # No shuffling in test data
        color_mode='rgb'
    )

    # Retrieve class labels
    classes = list(train_gen.class_indices.keys())
    print(f"Classes found: {classes}")

    return train_gen, valid_gen, test_gen, classes

# Example usage
main_dir = "/content/drive/MyDrive/Final_Project_Deep_Learning/split_data" # Main folder containing 'train', 'val', 'test'
img_size = (224, 224) # Image size for resizing
batch_size = 32 # Batch size for generators

train_gen, valid_gen, test_gen, classes = create_generators(main_dir, img_size, batch_size)

# Example: Display generator details
print(f"Number of classes: {len(classes)}")
print(f"Class labels: {classes}")

↗ Creating train generator...
Found 2470 images belonging to 3 classes.
Creating validation generator...
Found 529 images belonging to 3 classes.
Creating test generator...
Found 531 images belonging to 3 classes.
Classes found: ['Dolphin', 'Jelly Fish', 'Turtle_Tortoise']
Number of classes: 3
Class labels: ['Dolphin', 'Jelly Fish', 'Turtle_Tortoise']

```

NOTE : this cell might take little more time to **run**

✓ Plotting Augmented Images

```

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array

def visualize_four_augmentations(image_path, img_size):
    augmentations = [
        {"name": "Original Image", "gen": ImageDataGenerator()},
        {"name": "Horizontal Flip", "gen": ImageDataGenerator(horizontal_flip=True)},
        {"name": "Rotation 20°", "gen": ImageDataGenerator(rotation_range=20)},
        {"name": "Zoom 0.2", "gen": ImageDataGenerator(zoom_range=0.2)},
    ]

    # Load and preprocess the image
    img = load_img(image_path, target_size=img_size)
    img_array = img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) # Add batch dimension

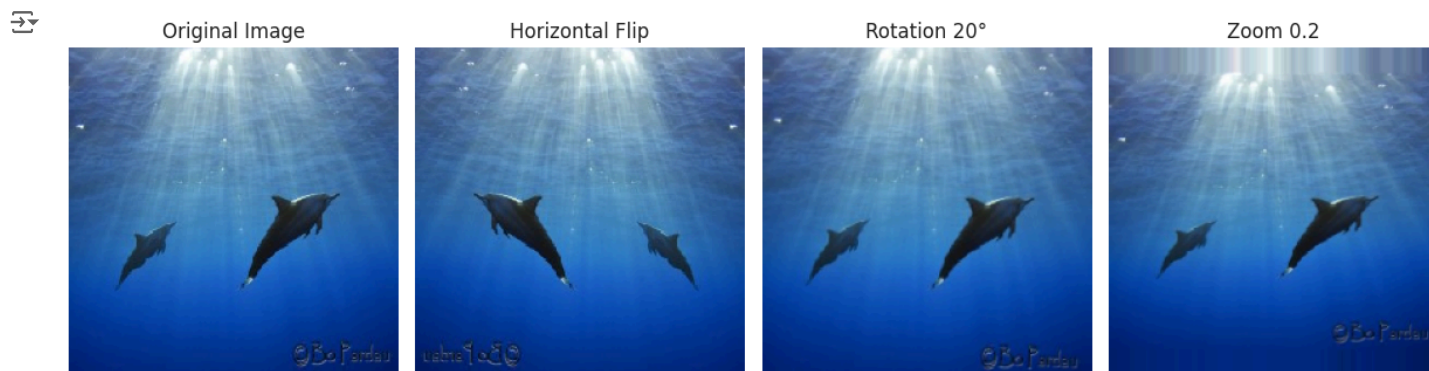
    # Plot the original image and augmentations
    plt.figure(figsize=(12, 6))
    for i, aug in enumerate(augmentations):
        aug_gen = aug["gen"].flow(img_array, batch_size=1) # Generate augmented image
        augmented_img = next(aug_gen)[0].astype('uint8') # Get augmented image

        plt.subplot(1, 4, i + 1)
        plt.imshow(augmented_img)
        plt.axis('off')
        plt.title(aug["name"]) # Set title to the name of the augmentation

    plt.tight_layout()
    plt.show()

```

```
image_path = '/content/drive/MyDrive/Final_Project_Deep_Learning/split_data/val/Dolphin/11331713213_37a9acd23c_b.jpg' # Path to a sample in
img_size = (224, 224) # Target image size
visualize_four_augmentations(image_path, img_size)
```



```
early_stop=EarlyStopping(monitor='val_loss',patience=3)
strategy = tf.distribute.MirroredStrategy()

with strategy.scope():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same', input_shape=(224, 224, 3)))
    model.add(BatchNormalization())

    # model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    #model.add(Dropout(0.2))

    model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
    model.add(BatchNormalization())
    # model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
    # model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    # model.add(Dropout(0.3))
    model.add(Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
    model.add(BatchNormalization())
    # model.add(Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'))
    # model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    # model.add(Dropout(0.4))
    model.add(Flatten())
    model.add(Dense(128, activation='relu', kernel_initializer='he_uniform'))
    model.add(BatchNormalization())
    model.add(Dropout(0.5))
    model.add(Dense(3, activation='softmax'))

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

callbacks = [
    ModelCheckpoint(
        filepath='best_model.keras', # Save the model in .h5 format
        monitor='val_loss',
        save_best_only=True, # Save only the best model
        mode='min', # Minimizing the validation loss
        verbose=1 # Print updates during training
    ),
    EarlyStopping(
        monitor='val_loss', # Monitor validation loss
        patience=5, # Stop training after 5 epochs of no improvement
        verbose=1, # Print when stopping
        mode='min' # Minimizing the validation loss
    )
]
```

```
print(model.summary())

epochs = 40
history=model.fit(x=train_gen, epochs=epochs, verbose=1, validation_data=valid_gen,
                  validation_steps=None, shuffle=True, initial_epoch=0)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization_10 (BatchNormalization)	(None, 224, 224, 32)	128
batch_normalization_11 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_6 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_7 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_12 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_7 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_8 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_13 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_8 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten_2 (Flatten)	(None, 100352)	0
dense_4 (Dense)	(None, 128)	12,845,184
batch_normalization_14 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 3)	387

Total params: 12,940,355 (49.36 MB)

Trainable params: 12,939,587 (49.36 MB)

Non-trainable params: 768 (3.00 KB)

None

```
/usr/local/lib/python3.10/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your `PyDataset` class
self._warn_if_super_not_called()
```

Epoch 1/40

78/78 ————— 62s 725ms/step - accuracy: 0.7661 - loss: 0.9205 - val_accuracy: 0.6106 - val_loss: 0.8276

Epoch 2/40

78/78 ————— 53s 617ms/step - accuracy: 0.8088 - loss: 0.5904 - val_accuracy: 0.7958 - val_loss: 0.5580

Epoch 3/40

78/78 ————— 60s 677ms/step - accuracy: 0.8334 - loss: 0.4571 - val_accuracy: 0.7089 - val_loss: 0.7974

Epoch 4/40

78/78 ————— 54s 637ms/step - accuracy: 0.8483 - loss: 0.4544 - val_accuracy: 0.8034 - val_loss: 0.5304

Epoch 5/40

78/78 ————— 60s 732ms/step - accuracy: 0.8470 - loss: 0.4269 - val_accuracy: 0.8110 - val_loss: 0.4507

Epoch 6/40

78/78 ————— 65s 783ms/step - accuracy: 0.8473 - loss: 0.4389 - val_accuracy: 0.8828 - val_loss: 0.3363

Epoch 7/40

78/78 ————— 82s 760ms/step - accuracy: 0.8699 - loss: 0.3689 - val_accuracy: 0.8639 - val_loss: 0.3781

Epoch 8/40

78/78 ————— 57s 666ms/step - accuracy: 0.8753 - loss: 0.3590 - val_accuracy: 0.8998 - val_loss: 0.2796

Epoch 9/40

78/78 ————— 47s 566ms/step - accuracy: 0.8858 - loss: 0.3643 - val_accuracy: 0.9055 - val_loss: 0.2409

Epoch 10/40

78/78 ————— 53s 644ms/step - accuracy: 0.8946 - loss: 0.3478 - val_accuracy: 0.9055 - val_loss: 0.2479

Epoch 11/40

78/78 ————— 80s 601ms/step - accuracy: 0.8939 - loss: 0.3106 - val_accuracy: 0.9149 - val_loss: 0.2419

Epoch 12/40

78/78 ————— 48s 569ms/step - accuracy: 0.8757 - loss: 0.3355 - val_accuracy: 0.8639 - val_loss: 0.3508

Epoch 13/40

78/78 ————— 82s 570ms/step - accuracy: 0.8773 - loss: 0.3191 - val_accuracy: 0.9112 - val_loss: 0.2605

Epoch 14/40

78/78 ————— 49s 558ms/step - accuracy: 0.8754 - loss: 0.3321 - val_accuracy: 0.9149 - val_loss: 0.2660

Epoch 15/40

78/78 ————— 48s 570ms/step - accuracy: 0.8889 - loss: 0.3159 - val_accuracy: 0.8941 - val_loss: 0.3104

Epoch 16/40

78/78 ————— 47s 566ms/step - accuracy: 0.9027 - loss: 0.2861 - val_accuracy: 0.9074 - val_loss: 0.2772

Epoch 17/40

78/78 ————— 53s 610ms/step - accuracy: 0.8856 - loss: 0.3177 - val_accuracy: 0.8904 - val_loss: 0.3381

Epoch 18/40

78/78 ————— 48s 574ms/step - accuracy: 0.8902 - loss: 0.2981 - val_accuracy: 0.9149 - val_loss: 0.2635

Epoch 19/40

78/78 ————— 48s 567ms/step - accuracy: 0.8855 - loss: 0.3093 - val_accuracy: 0.8601 - val_loss: 0.4645

Epoch 20/40

78/78 ————— 51s 587ms/step - accuracy: 0.9008 - loss: 0.2776 - val_accuracy: 0.8752 - val_loss: 0.3172

```

Epoch 21/40
78/78 ————— 46s 548ms/step - accuracy: 0.8923 - loss: 0.2943 - val_accuracy: 0.9093 - val_loss: 0.2489
Epoch 22/40
78/78 ————— 47s 568ms/step - accuracy: 0.8924 - loss: 0.3157 - val_accuracy: 0.9319 - val_loss: 0.2150
Epoch 23/40
78/78 ————— 51s 596ms/step - accuracy: 0.9053 - loss: 0.2707 - val_accuracy: 0.8979 - val_loss: 0.2793
Epoch 24/40
78/78 ————— 82s 608ms/step - accuracy: 0.9018 - loss: 0.2843 - val_accuracy: 0.8979 - val_loss: 0.2975
Epoch 25/40
78/78 ————— 51s 613ms/step - accuracy: 0.9068 - loss: 0.2702 - val_accuracy: 0.9017 - val_loss: 0.2622
Epoch 26/40
78/78 ————— 50s 597ms/step - accuracy: 0.9218 - loss: 0.2451 - val_accuracy: 0.9036 - val_loss: 0.3037
Epoch 27/40
78/78 ————— 50s 604ms/step - accuracy: 0.8984 - loss: 0.2913 - val_accuracy: 0.8998 - val_loss: 0.2509
Epoch 28/40
78/78 ————— 50s 599ms/step - accuracy: 0.9163 - loss: 0.2578 - val_accuracy: 0.8809 - val_loss: 0.3019
Epoch 29/40
78/78 ————— 50s 596ms/step - accuracy: 0.9039 - loss: 0.2648 - val_accuracy: 0.9036 - val_loss: 0.2564
Epoch 30/40
78/78 ————— 48s 564ms/step - accuracy: 0.8899 - loss: 0.2981 - val_accuracy: 0.9282 - val_loss: 0.2222
Epoch 31/40
78/78 ————— 46s 537ms/step - accuracy: 0.9070 - loss: 0.2637 - val_accuracy: 0.9244 - val_loss: 0.2409
Epoch 32/40
78/78 ————— 86s 586ms/step - accuracy: 0.9104 - loss: 0.2631 - val_accuracy: 0.9017 - val_loss: 0.2517
Epoch 33/40
78/78 ————— 47s 563ms/step - accuracy: 0.9031 - loss: 0.2638 - val_accuracy: 0.8733 - val_loss: 0.3342
Epoch 34/40
78/78 ————— 54s 658ms/step - accuracy: 0.9120 - loss: 0.2407 - val_accuracy: 0.9357 - val_loss: 0.2122
Epoch 35/40
78/78 ————— 45s 541ms/step - accuracy: 0.9057 - loss: 0.2694 - val_accuracy: 0.9055 - val_loss: 0.2432
Epoch 36/40
78/78 ————— 46s 546ms/step - accuracy: 0.9137 - loss: 0.2401 - val_accuracy: 0.9187 - val_loss: 0.2441
Epoch 37/40
78/78 ————— 50s 604ms/step - accuracy: 0.9195 - loss: 0.2300 - val_accuracy: 0.9301 - val_loss: 0.2124
Epoch 38/40
78/78 ————— 46s 545ms/step - accuracy: 0.9178 - loss: 0.2215 - val_accuracy: 0.8658 - val_loss: 0.4333
Epoch 39/40
78/78 ————— 48s 569ms/step - accuracy: 0.9157 - loss: 0.2322 - val_accuracy: 0.9093 - val_loss: 0.2447
Epoch 40/40
78/78 ————— 51s 612ms/step - accuracy: 0.9209 - loss: 0.2039 - val_accuracy: 0.9187 - val_loss: 0.2556

```

```

# Call the function to plot
plot_training_history(history)

```



Training and Validation Loss

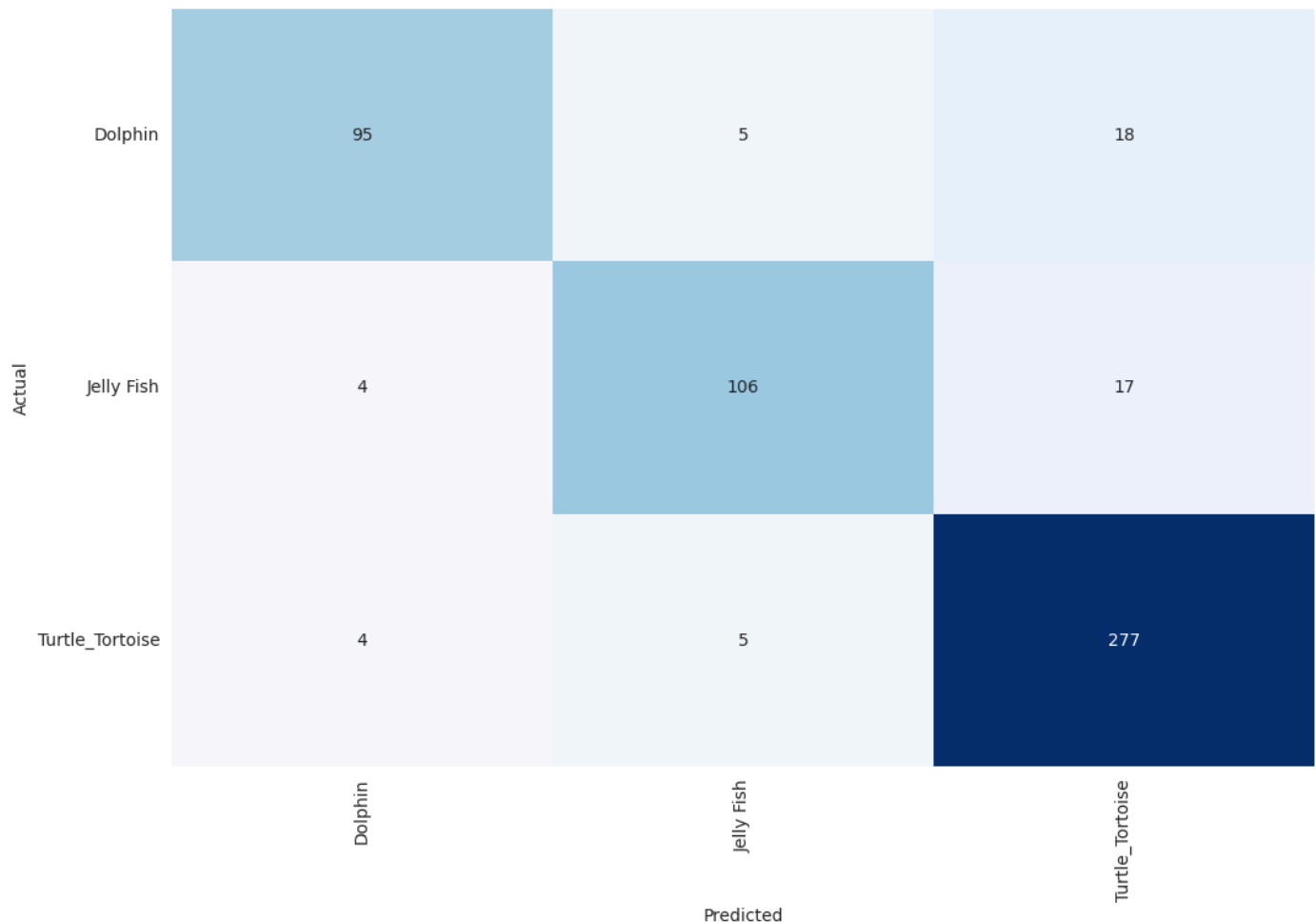


✓ Evaluating The Model

```
errors, tests, error_list, f1score =evaluate(test_gen)
```

17/17 3s 150ms/step
there were 53 errors in 531 tests for an accuracy of 90.02

Confusion Matrix



Classification Report:

	precision	recall	f1-score	support
Dolphin	0.9223	0.8051	0.8597	118
Jelly Fish	0.9138	0.8346	0.8724	127
Turtle_Tortoise	0.8878	0.9685	0.9264	286
accuracy			0.9002	531
macro avg	0.9080	0.8694	0.8862	531
weighted avg	0.9017	0.9002	0.8987	531

```
working_dir=r'/content/drive/MyDrive/Colab Notebooks/Final_Project_Deep_Learning'
subject='sea animals-efficient-'
save_id=f'{subject}-{f1score:5.2f}.h5'
model_save_loc=os.path.join(working_dir, save_id)
model.save(model_save_loc)
msg= f'model was saved as {model_save_loc}'
print(msg)
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend you to use the new format via `model.save(filepath, save\_format='tf')` or `keras.saving.save\_model(model, filepath, save\_format='tf')`.

Start coding or [generate](#) with AI.

✓ Transfer Learning

- Training on Custom Data Using EfficientNetB7